ARCHITECTURE DECISION RECORD
VERSION 1.1

YADA
(YET ANOTHER DELIVERY APP)

JULY 2026

TABLE OF CONTENTS

ADR-001: USE SVELTEKIT AS THE FRONTEND FRAMEWORK

Status: Accepted
Date: 14 July 2026
Deciders: YADA project team
Supersedes: None

Context

YADA requires a fast, responsive web interface for two user groups: business staff who dispatch and monitor deliveries ("Dispatchers") and the delivery personnel who fulfil them ("Riders"). The frontend must provide an intuitive user experience, support real-time updates, integrate with mapping services, and keep the client bundle small.

Decision

The project will use SvelteKit as the frontend framework, Tailwind CSS for styling, and the Google Maps API for displaying Rider locations and delivery destinations. SvelteKit was selected for its rendering speed and its development philosophy of compiling away framework overhead rather than shipping a runtime to the browser.

Alternatives Considered

React with Next.js offers the largest community and hiring pool, but it ships a heavier client runtime and requires more boilerplate for the same screens. Rejected: bundle size and ceremony outweighed the ecosystem advantage for a small team.
Vue with Nuxt offers comparable developer experience and a gentler learning curve. Rejected: no decisive advantage over SvelteKit for this workload, and the team had no existing Vue experience to leverage.
Mapbox GL or Leaflet with OpenStreetMap would give lower licensing cost and full control over tiles. Rejected in favour of the Google Maps API for its geocoding and Places coverage in the operating region, which the delivery-address workflow depends on.

Consequences

Advantages

1. High performance with minimal JavaScript bundle size.
2. File-based routing simplifies navigation.
3. Tailwind CSS enables rapid and consistent UI development.
4. Easy integration with Google Maps and real-time updates.

Disadvantages

1. Smaller developer community than React.

ADR-002: USE SVELTEKIT SERVER ROUTES WITH TYPESCRIPT FOR BACKEND DEVELOPMENT

Status: Accepted
Date: 14 July 2026
Deciders: YADA project team
Supersedes: None

Context

The backend must manage authentication, delivery requests, Rider assignments, real-time communication, and API endpoints while maintaining code quality and scalability.

Decision

The backend will be implemented as SvelteKit server routes and endpoints written in TypeScript, deployed to Cloudflare Workers through adapter-cloudflare. Frontend and backend therefore share one codebase, one build pipeline, and one set of type definitions, and the application requires no separately hosted API service.

Alternatives Considered

A standalone Node.js service alongside the SvelteKit app would provide clean separation between interface and API. Rejected: SvelteKit already provides server routes, so a second service would add a deployment target, a network hop, and a second hosting provider without a corresponding benefit at this scale.
Python with FastAPI has a strong ecosystem for data work. Rejected: a second language would split the team's context and forfeit end-to-end type sharing with the frontend.
Go offers better raw concurrency. Rejected: slower iteration for a project of this size, and real-time coordination is handled by Durable Objects under ADR-006 rather than by the API tier.
Deno or Bun would remove the compilation step through native TypeScript execution. Rejected: neither is needed once the application targets the Workers runtime, and hosting support is less mature than Cloudflare's.

Consequences

Advantages

1. Strong type safety reduces runtime errors.
2. Improved maintainability and code readability.
3. Efficient asynchronous processing.
4. Large ecosystem and excellent tooling.

Disadvantages

1. Additional compilation step.
2. The Workers runtime is not Node.js: libraries that depend on Node APIs or raw TCP sockets must be replaced with Workers-compatible equivalents, including Neon's HTTP driver in place of node-postgres.
3. CPU time per request is bounded by Workers limits, so long-running work must be moved to queues or scheduled jobs.

ADR-003: USE POSTGRESQL WITH DRIZZLE ORM AND NEON CONSOLE

Status: Accepted
Date: 14 July 2026
Deciders: YADA project team
Supersedes: None

Context

YADA stores structured information such as Riders, delivery requests, authentication data, and delivery history. A reliable relational database with cloud hosting is required.

Decision

The project will use PostgreSQL as the primary database, Drizzle ORM for database access and schema management, and Neon Console as the serverless PostgreSQL hosting platform. Because the backend runs on Cloudflare Workers (ADR-002), database access will use Neon's HTTP/serverless driver rather than a TCP connection pool.

Alternatives Considered

MongoDB offers a flexible schema and simple horizontal scaling. Rejected: delivery, Rider, and payment records are strongly relational, and the application depends on joins and referential integrity.
MySQL offers comparable maturity and hosting options. Rejected: PostgreSQL has stronger serverless support on Neon, and Drizzle's PostgreSQL dialect is the most mature part of the ORM.
Prisma, used instead of Drizzle, would bring richer tooling and a larger user base. Rejected: Drizzle's SQL-first API and lighter runtime suit serverless execution on Neon, where Prisma's query engine adds cold-start cost.
Supabase, used instead of Neon, bundles authentication and storage with PostgreSQL. Rejected: authentication is decided separately in ADR-004, so the bundled features would duplicate that choice.

Consequences

Advantages

1. ACID transactions and constraint enforcement protect delivery and payment records.
2. Mature query planner and indexing support the reporting workload.
3. Type-safe queries through Drizzle ORM.
4. Neon provides scalable cloud-hosted PostgreSQL.

Disadvantages

1. Requires careful database schema design.
2. Internet connectivity is required to access the hosted database.
3. Scale-to-zero on Neon introduces cold-start latency on the first query after an idle period.

ADR-004: USE BETTER AUTH WITH GOOGLE OAUTH AND PASSWORD AUTHENTICATION

Status: Accepted
Date: 14 July 2026
Deciders: YADA project team
Supersedes: None

Context

The application requires secure authentication for Dispatchers and Riders while supporting both traditional login and third-party authentication.

Decision

The project will use Better Auth as the authentication framework, with Google as the OAuth provider for social sign-in and traditional email/password authentication as the fallback. User and session records are stored in the primary PostgreSQL database (ADR-003) through Drizzle.

Alternatives Considered

Auth0 or Clerk would provide managed authentication with minimal implementation effort. Rejected: per-user pricing scales poorly against a Rider fleet, and user records would sit outside the primary database.
Auth.js is the incumbent open-source option for this ecosystem. Rejected: less complete first-party support for email/password flows and session management than Better Auth.
A custom implementation would give full control over session and token handling. Rejected: authentication defects are high-severity and a bespoke implementation would carry that risk without compensating benefit.

Consequences

Advantages

1. Vetted session handling, password hashing, and CSRF protection by default.
2. Supports OAuth and email/password flows behind one interface.
3. Simplifies session and user management.
4. Easily integrates with modern web applications.

Disadvantages

1. Initial configuration is more complex.
2. Google OAuth requires a Google Cloud project, consent-screen configuration, and separate credentials per environment.

ADR-005: DEPLOY ON CLOUDFLARE WITH NEON-HOSTED POSTGRESQL

Status: Accepted
Date: 14 July 2026
Deciders: YADA project team
Supersedes: None

Context

The application must be deployed to a reliable cloud platform for client demonstrations and production readiness.

Decision

Frontend and backend will both be deployed on Cloudflare as a single Workers deployment covering static assets and SvelteKit server routes, with DNS, TLS, and WAF protection managed in the same Cloudflare account and real-time coordination provided by Durable Objects under ADR-006. The PostgreSQL database is hosted separately on Neon's serverless platform under ADR-003. Cloudflare and Neon are therefore the only two vendors in the deployment.

Alternatives Considered

Railway or Render would host a separate backend service as a persistent Node.js process. Rejected: once the backend runs as SvelteKit server routes on Workers, a second provider adds cost and operational surface with no corresponding benefit.
Vercel is the most direct hosting path for SvelteKit. Rejected: DNS and edge security would still require a separate provider, and Durable Objects are available only on Cloudflare.
AWS with ECS or Amplify offers the most control and the widest service range. Rejected: operational overhead disproportionate to a project at this stage, and no team member holds AWS operations experience.

Consequences

Advantages

1. Automatic deployment from GitHub.
2. Fast global content delivery through Cloudflare.
3. Frontend, backend, DNS, and edge security run under one account and one deployment pipeline.
4. Easy environment variable management.

Disadvantages

1. Free-tier resource, build-minute, and bandwidth limits.
2. Requires internet connectivity for deployment and access.
3. Concentrating delivery, compute, and real-time coordination on Cloudflare increases vendor lock-in; migration would affect ADR-002 and ADR-006 as well as this record.

ADR-006: USE CLOUDFLARE DURABLE OBJECTS FOR REAL-TIME LOCATION TRACKING

Status: Accepted
Date: 14 July 2026
Deciders: YADA project team
Supersedes: None

Context

Dispatchers must monitor Rider locations and delivery progress in real time to improve dispatch efficiency and operational visibility.

Decision

Real-time tracking will be implemented with Cloudflare Durable Objects. One Durable Object instance per active delivery holds the Rider's current position and streams updates to the Dispatcher dashboard over WebSockets, using the hibernation API so that idle connections consume no billable resources. Position snapshots and completed delivery trails are persisted to the Neon-hosted PostgreSQL database (ADR-003) as latitude and longitude columns; map rendering and route display use the Google Maps API (ADR-001).

Alternatives Considered

Socket.IO on a Node.js host is the conventional choice for bidirectional messaging. Rejected: it requires an always-on server outside Cloudflare, reintroducing the second hosting provider that ADR-005 removes, and connection state would have to be shared across instances.

PostGIS with server-side spatial queries provides native geospatial types and indexes. Rejected: live tracking is a streaming problem rather than a query problem. Current position lives in the Durable Object, and the historical trail requires only latitude and longitude columns, so the extension dependency is not justified.
HTTP polling is the simplest to implement and requires no persistent connections. Rejected: either latency or request volume becomes unacceptable as the Rider fleet grows.
Server-Sent Events are lighter than WebSockets for one-way streaming. Rejected: Riders also send position updates upstream, so a bidirectional channel is required.

Consequences

Advantages

1. Real-time coordination runs on the same platform as the frontend and backend, with no additional hosting provider.
2. Each Durable Object is a single-threaded, strongly consistent coordination point for one delivery, which removes the need to share connection state across instances.
3. WebSocket hibernation keeps idle Rider connections inexpensive as the fleet grows.
4. Durable Object storage survives restarts, so in-flight delivery state is not lost on redeployment.

Disadvantages

1. Durable Objects are specific to Cloudflare; moving off the platform would require re-implementing real-time coordination.
2. Durable Objects require a paid Workers plan.
3. Local development depends on Wrangler and Miniflare emulation rather than a plain Node.js process.
4. Historical location analysis must be served from PostgreSQL, because Durable Object storage cannot be queried across instances.

ADR-007: LICENSE THE PROJECT UNDER APACHE-2.0

Status: Accepted
Date: 20th August, 2026
Deciders: YADA project team
Supersedes: None

Context

YADA is developed in a public repository. Without a declared licence, default copyright applies and no third party may legally copy, modify, or deploy the code, which blocks review, reuse, and any future handover to a client. A licence must therefore be chosen and recorded before the

repository is shared beyond the team. The application also carries obligations inherited from others. It inlines icons from three sets, loads the Google Maps Platform SDK at runtime under that platform's terms, and embeds a small table of coordinates read from OpenStreetMap, which is published under a share-alike data licence. Each of these places a condition on distribution that the licensing decision has to account for rather than assume away.

Decision

The project is licensed under the Apache License, Version 2.0. The unmodified licence text is committed as LICENSE at the repository root and covers all source code, principally everything under App/.

The documentation and design assets in Docs/ and Design/ are licensed CC BY 4.0 instead, recorded in LICENSE-DOCS. That file states the scope explicitly, links the deed and the legal code, and notes that source code is not covered by it. Creative Commons does not require the full legal code to be copied into a work, so the notice and the link are how the licence is applied. Attribution is split across two files, and the split is deliberate:

- NOTICE carries only what must propagate. Anything placed in a NOTICE file must be reproduced by every downstream distributor under Apache-2.0 §4(d), so it holds the OpenStreetMap attribution and nothing else, together with a note that Google Maps attribution is rendered on screen by the SDK.
- THIRD-PARTY-NOTICES.md carries dependency licences, scoped to what actually ships: the browser bundle and the Cloudflare Worker bundle. Build and development tooling is not distributed and is not listed.

Third-party dependencies retain their own licences. The Apache-2.0 grant covers only code authored by the project team.

Alternatives Considered

1. MIT is shorter and is the most widely recognised permissive licence. Rejected: it contains no express patent grant and no contribution terms, both of which Apache-2.0 provides at no practical cost to adopters.
2. BSD-3-Clause is permissive and adds a non-endorsement clause. Rejected: equivalent in effect to MIT for this project, and likewise silent on patents.
3. Proprietary licensing, with all rights reserved, would give maximum control over reuse. Rejected: the repository is public and the team intends the work to be reviewable and reusable.
4. AGPL-3.0 is a strong copyleft licence extending to network use, which would compel anyone hosting a modified copy to publish source. Rejected: it would deter adoption and complicate any future commercial deployment of YADA.
5. A single licence across the whole repository, with Apache-2.0 applied to documentation and design as well as code, would be simpler to state and to police. Rejected: Apache-2.0 is written for software and its §4 source-and-object distinctions do not map onto an ERD,

a screenshot, or a token file, whereas CC BY is the recognised instrument for that material and is what the Solar icon set already arrives under.

Consequences

Advantages

1. Contributors grant an express patent licence, which removes a class of legal risk that MIT and BSD leave open.
2. Section 5 states that contributions are made under the same terms, so inbound contributions need no separate agreement, which matters for a multi-author project with no CLA.
3. Permissive terms allow commercial use and closed-source deployment, keeping a future client handover straightforward.
4. Apache-2.0 is routinely cleared by corporate legal review, which lowers the barrier to adoption.

Disadvantages

1. LICENSE, LICENSE-DOCS, NOTICE and THIRD-PARTY-NOTICES.md must be kept accurate as dependencies change. Adding a runtime dependency is now also a documentation change.
2. Permissive terms allow forks to be commercialised without contributing changes back.
3. Apache-2.0 is incompatible with GPLv2, which would exclude any future GPLv2-only dependency.
4. Two licences in one repository means contributors have to know which applies to what. The boundary is drawn by directory, and LICENSE-DOCS states it in both directions, but it is a boundary that did not previously exist.

TECHNOLOGY STACK SUMMARY

Layer	Technology
Frontend	SvelteKit, Tailwind CSS
Backend	SvelteKit server routes on Cloudflare Workers, TypeScript
Real-Time Communication	Cloudflare Durable Objects (WebSockets with hibernation)
Location Services	Google Maps API (mapping, geocoding); latitude/longitude in PostgreSQL
Database	PostgreSQL, Drizzle ORM, Neon Console (serverless)
Authentication	Better Auth, Google OAuth, Email/Password
Deployment	Cloudflare (frontend and backend), Neon Console (database)